

Evidencias EV08 — FastAPI Intermedio (CRUD, errores, Swagger y Depends)

Guia: GA1-220501096-01-AA1-EV08 **Aprendiz:** Daniel Roman **Proyecto:** device_systems (v2.0.0) — código en `../device_systems/` **Repositorio:** <https://github.com/daniel-roman345/python-avanzado->

1. Documentación automática

1.1 Swagger UI (/docs)

Endpoints organizados con los tags **Users** y **Root**, cada uno con `summary`, `description` y los códigos de respuesta posibles.

device_systems API 2.0.0 OAS 3.1

/openapi.json

API REST para la gestion de usuarios del sistema **device_systems**.

Caracteristicas de esta version

- CRUD completo: [GET](#) , [POST](#) , [PUT](#) , [PATCH](#) , [DELETE](#) .
- Manejo de errores con [HTTPException](#) y respuestas estructuradas.
- Codigos de estado HTTP correctos en cada operacion.
- Dependencias reutilizables con [Depends\(\)](#) .
- Documentacion automatica en [/docs](#) y [/redoc](#) .

Contact [Daniel Roman](#) - Aprendiz SENA

Users CRUD completo del recurso **usuarios**: listar, consultar, crear, actualizar (PUT y PATCH) y eliminar.

GET	/users	Listar usuarios	⌵
POST	/users	Crear un usuario	⌵
GET	/users/{user_id}	Consultar usuario por ID	⌵
PUT	/users/{user_id}	Actualizar un usuario por completo	⌵
PATCH	/users/{user_id}	Actualizar parcialmente un usuario	⌵
DELETE	/users/{user_id}	Eliminar un usuario	⌵

Root Informacion general y estado de la API.

GET	/	Informacion de la API	⌵
GET	/health	Estado del servicio	⌵

Schemas			⌵
HTTPValidationError	>	Expand all	object
MessageResponse	>	Expand all	object
UserCreate	>	Expand all	object
UserListResponse	>	Expand all	object
UserPatch	>	Expand all	object
UserResponse	>	Expand all	object
UserRole	>	Expand all	string
UserUpdate	>	Expand all	object
ValidationError	>	Expand all	object

1.2 ReDoc (/redoc)

[illegible]

2. Evidencia de pruebas de cada endpoint

2.1 GET /users

GET /users

Listar usuarios

Lista los usuarios registrados. Admite los query parameters **role** (admin, support, user) e **is_active** (true/false).

Parameters

Cancel

Name	Description
is_active	Filtra por estado activo o inactivo
boolean (boolean null) (query)	--
role	
string (string null) (query)	role

ExecuteClear

Responses

Curl

curl -X 'GET' \ 'http://127.0.0.1:8088/users' \ -H 'accept: application/json'

Request URL

http://127.0.0.1:8088/users

Server response

CodeDetails

200

Response body

```
{
  "total": 3,
  "data": [
    {
      "name": "Ana Perez",
      "email": "ana@ena.edu.co",
      "role": "admin",
      "is_active": true,
      "id": 1
    },
    {
      "name": "Carlos Gomez",
      "email": "carlos@ena.edu.co",
      "role": "support",
      "is_active": true,
      "id": 2
    },
    {
      "name": "Laura Martinez",
      "email": "laura@ena.edu.co",
      "role": "user",
      "is_active": false,
      "id": 3
    }
  ]
}
```

Response headers

```
content-length: 293
content-type: application/json
date: Sat, 12 Sep 2026 17:47:58 GMT
server: uvicorn
x-api-version: 2.0
x-app-name: device_systems
x-total-users: 3
```

Responses

Code	Description	Links
200	Listado de usuarios encontrados	No links
Media type		
application/json		
Controls Accept header		
Example Value Schema		
<pre>{ "total": 0, "data": [{ "name": "Ana Perez", "email": "ana@ena.edu.co", "role": "admin", "is_active": true, "id": 0 }] }</pre>		
400	Solicitud invalida	No links
422	Validation Error	No links
Media type		
application/json		
Example Value Schema		
<pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>		

2.2 GET /users/{user_id}

GET

/users/{user_id}

Consultar usuario por ID

Obtiene un usuario por su identificador. Usa la dependencia `get_user_or_404`, que lanza un 404 si el usuario no existe.

Parameters

Cancel

Name	Description
user_id integer (path) required minimum: 1	Identificador del usuario

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8008/users/2' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8008/users/2
```

Server response

Code	Details
200	Response body <pre>{ "name": "Carlos Gomez", "email": "carlos@sena.edu.co", "role": "support", "is_active": true, "id": 2 }</pre> Response headers <pre>content-length: 93 content-type: application/json date: Sat, 12 Sep 2026 17:48:02 GMT server: uvicorn x-api-version: 2.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
200	Usuario encontrado Media type application/json Controls Accept header. Example Value Schema <pre>{ "name": "Ana Perez", "email": "ana@sena.edu.co", "role": "admin", "is_active": true, "id": 0 }</pre>	No links
404	Usuario no encontrado	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

2.3 POST /users — 201 Created

POST /users

Crear un usuario

Registra un usuario nuevo. La dependencia `validar_email_unico` impide correos duplicados.

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

Edit Value

Schema

```
{
  "name": "Pedro Ruiz",
  "email": "pedro@sena.edu.co",
  "role": "user",
  "is_active": true
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
'http://127.0.0.1:8088/users' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "name": "Pedro Ruiz",
  "email": "pedro@sena.edu.co",
  "role": "user",
  "is_active": true
}'
```

Request URL

http://127.0.0.1:8088/users

Server response

Code

Details

201

Response body

```
{
  "name": "Pedro Ruiz",
  "email": "pedro@sena.edu.co",
  "role": "user",
  "is_active": true,
  "id": 4
}
```

Download

Response headers

```
content-length: 87
content-type: application/json
date: Sat, 12 Sep 2026 17:48:04 GMT
location: /users/4
server: uvicorn
x-api-version: 2.0
x-app-name: device_systems
```

Responses

Code	Description	Links
201	Usuario creado correctamente	No links
Media type application/json Controls Accept header. Example ValueSchema<pre>{ "name": "Ana Perez", "email": "ana@sena.edu.co", "role": "admin", "is_active": true, "id": 0 }</pre>		
400	Solicitud invalida	No links
422	Datos invalidos	No links

2.4 PUT /users/{user_id} — actualizacion completa, 200 OK

PUT

/users/{user_id} Actualizar un usuario por completo

Reemplaza toda la informacion del usuario. Requiere enviar `name`, `email`, `role` e `is_active`.

Parameters

Cancel

Reset

Name	Description
user_id required	Identificador del usuario
integer (path)	2
	minimum: 1

Request body required

application/json

Edit Value

Schema

```
{  "name": "Carlos A Gomez",  "email": "carlos.g@sena.edu.co",  "role": "admin",  "is_active": false}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \  'http://127.0.0.1:8080/users/2' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "name": "Carlos A Gomez",  "email": "carlos.g@sena.edu.co",  "role": "admin",  "is_active": false}'
```

Request URL

```
http://127.0.0.1:8080/users/2
```

Server response

Code	Details
200	Response body <pre>{ "name": "Carlos A Gomez", "email": "carlos.g@sena.edu.co", "role": "admin", "is_active": false, "id": 2}</pre> Download Response headers <pre>content-length: 96 content-type: application/json date: Sat, 11 Sep 2026 17:48:15 GMT server: uvicorn x-api-version: 2.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
200	Usuario actualizado	No links
	Media type	
	application/json Controla Accept header.	
	Example Value	Schema
	<pre>{ "name": "Ana Perez", "email": "ana@sena.edu.co", "role": "admin", "is_active": true, "id": 0}</pre>	
400	Solicitud invalida	No links
404	Usuario no encontrado	No links
422	Validation Error	No links
	Media type	
	application/json	
	Example Value	Schema
	<pre>{ "detail": { "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }}</pre>	

2.5 PATCH /users/{user_id} — actualizacion parcial, 200 OK

Solo se envio `{"role": "support"}` y el resto de los campos se conservo.

PATCH

/users/{user_id}

Actualizar parcialmente un usuario

Modifica solo los campos enviados. Si el cuerpo llega vacío la dependencia `validar_patch_no_vacio` responde 400 Bad Request.

Parameters

Cancel

Name	Description
user_id required	Identificador del usuario
integer	
(path)	
3	
minimum: 1	

Request body required

application/json

Edit Value | Schema

```
{  "role": "support"}
```

Execute

Clear

Responses

Curl

```
curl -X 'PATCH' \
  'http://127.0.0.1:8080/users/3' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "role": "support"
  }'
```

Request URL

http://127.0.0.1:8080/users/3

Server response

Code	Details
200	Response body <pre>{ "name": "Laura Martinez", "email": "laure@demu.edu.co", "role": "support", "is_active": false, "id": 3}</pre> Download Response headers <pre>content-length: 95 content-type: application/json date: Sat, 12 Sep 2026 17:48:18 GMT server: uvicorn x-api-version: 2.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
200	Usuario actualizado parcialmente	No links
Media type application/json Controla Accept header. Example Value Schema <pre>{ "name": "Ana Perez", "email": "amagena.edu.co", "role": "admin", "is_active": true, "id": 0}</pre>		
400	Solicitud invalida	No links
404	Usuario no encontrado	No links
422	Validation Error	No links
Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }]}</pre>		

2.6 DELETE /users/{user_id} — 200 OK con mensaje

DELETE

/users/{user_id}

Eliminar un usuario

Elimina un usuario existente y confirma la operacion con un mensaje. Si el usuario no existe responde 404 Not Found.

Parameters

Cancel

Name	Description
user_id * required	Identificador del usuario
integer (path)	4 minimum: 1

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/users/4' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/users/4
```

Server response

Code	Details
200	Response body <pre>{ "message": "Usuario 4 eliminado correctamente" }</pre> Download Response headers <pre>content-length: 47 content-type: application/json date: Sat, 12 Sep 2026 17:48:28 GMT server: uvicorn x-api-version: 2.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
200	Usuario eliminado correctamente Media type application/json Controls Accept header. Example Value Schema <pre>{ "message": "Usuario eliminado correctamente" }</pre>	No links
404	Usuario no encontrado	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

3. Evidencia de errores controlados

3.1 Usuario no encontrado — 404

GET

/users/{user_id}

Consultar usuario por ID

Obtiene un usuario por su identificador. Usa la dependencia `get_user_or_404`, que lanza un 404 si el usuario no existe.

Parameters

Cancel

Name	Description
user_id ★ required	Identificador del usuario
integer (path)	99 minimum: 1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8008/users/99' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8008/users/99
```

Server response

Code	Details
404	Error: Not Found Response body <pre>{ "error": true, "message": "Usuario no encontrado", "status_code": 404, "path": "/users/99" }</pre> Download Response headers <pre>content-length: 85 content-type: application/json date: Sat, 12 Sep 2026 17:48:25 GMT server: uvicorn x-api-version: 2.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
200	Usuario encontrado Media type application/json Controls Accept header. Example Value Schema <pre>{ "name": "Ana Perez", "email": "ana@sena.edu.co", "role": "admin", "is_active": true, "id": 0 }</pre>	No links
404	Usuario no encontrado	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

3.2 PATCH sin datos — 400

PATCH

/users/{user_id} Actualizar parcialmente un usuario

⌵

Modifica solo los campos enviados. Si el cuerpo llega vacío la dependencia `validar_patch_no_vacio` responde 400 Bad Request.

Parameters

Cancel

Reset

Name	Description
user_id * required	Identificador del usuario
integer (path)	3
	minimum: 1

Request body required

application/json

Edit Value

Schema

{}

Execute

Clear

Responses

Curl

curl -X 'PATCH' \n http://127.0.0.1:8008/users/3' \n -H 'accept: application/json' \n -H 'Content-Type: application/json' \n -d {}

Request URL

http://127.0.0.1:8008/users/3

Server response

Code	Details
400	Error: Bad Request Response body {\n "error": true,\n "message": "Debe enviar al menos un campo para actualizar",\n "status_code": 400,\n "path": "/users/3"\n} Response headers content-length: 108\ncontent-type: application/json\ndate: Sat, 12 Sep 2020 17:48:22 GMT\nserver: uvicorn\nx-api-version: 2.0\nx-app-name: device_systems

Responses

Code	Description	Links
200	Usuario actualizado parcialmente	No links
400	Solicitud invalida	No links
404	Usuario no encontrado	No links
422	Validation Error	No links

Media type

application/json

Controls Accept header.

Example Value

Schema

{\n "name": "Ana Perez",\n "email": "ana@sona.edu.co",\n "role": "admin",\n "is_active": true,\n "id": 0\n}

Media type

application/json

Example Value

Schema

{\n "detail": [\n {\n "loc": [\n "string",\n 0\n],\n "msg": "string",\n "type": "string",\n "input": "string",\n "ctx": {}\n }\n]\n}

3.3 Correo duplicado — 400

POST

/users

Crear un usuario

Registra un usuario nuevo. La dependencia `validar_email_unico` impide correos duplicados.

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

Edit Value

Schema

```
{
  "name": "Pedro Ruiz",
  "email": "pedro@sena.edu.co",
  "role": "user",
  "is_active": true
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8008/users' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Pedro Ruiz",
    "email": "pedro@sena.edu.co",
    "role": "user",
    "is_active": true
  }'
```

Request URL

http://127.0.0.1:8008/users

Server response

Code

Details

400

Error: Bad Request

Response body

```
{
  "error": true,
  "message": "El correo ya esta registrado",
  "status_code": 400,
  "path": "/users"
}
```

Response headers

```
content-length: 89
content-type: application/json
date: Sat, 12 Sep 2026 17:48:08 GMT
server: uvicorn
x-api-version: 2.0
x-app-name: device_systems
```

Responses

Code	Description	Links
201	Usuario creado correctamente	No links
Media type		
application/json		
Controls Accept header.		
Example Value Schema		
<pre>{ "name": "Ana Perez", "email": "ana@sena.edu.co", "role": "admin", "is_active": true, "id": 0 }</pre>		
400	Solicitud invalida	No links
422	Datos invalidos	No links

3.4 Datos invalidos — 422

POST /users Crear un usuario

Registra un usuario nuevo. La dependencia `validar_email_unico` impide correos duplicados.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value

Schema

```
{  "name": "ab",  "email": "maio",  "role": "root",  "is_active": true}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  http://127.0.0.1:8000/users' \  -H 'accept: application/json' \  -H 'Content-type: application/json' \  -d '{  "name": "ab",  "email": "maio",  "role": "root",  "is_active": true}'
```

Request URL

http://127.0.0.1:8000/users

Server response

Code	Details
422	Error: Unprocessable Content

Response body

```
{  "error": true,  "message": "Error de validacion en los datos enviados",  "status_code": 422,  "detail": {    "campo": "body.name",    "mensaje": "String should have at least 3 characters"  },  "campo": "body.email",  "mensaje": "value is not a valid email address: An email address must have an @-sign."  },  "campo": "body.role",  "mensaje": "Input should be 'admin', 'support' or 'user'"  } }
```

Download

Response headers

```
content-length: 260  content-type: application/json  date: Sat, 12 Sep 2026 17:48:12 GMT  server: uvicorn  x-api-version: 2.0  x-app-name: device_systems
```

Responses

Code	Description	Links
201	Usuario creado correctamente	No links
	Media type application/json Controls Accept header: Example Value Schema <pre>{ "name": "Ana Perez", "email": "ana@sena.edu.co", "role": "admin", "is_active": true, "id": 0}</pre>	
400	Solicitud invalida	No links
422	Datos invalidos	No links

Todas las respuestas de error usan el mismo formato gracias al `exception_handler` registrado en `app/main.py` :

```
{ "error": true, "message": "Usuario no encontrado", "status_code": 404, "path": "/users/99" }
```

Pagina 13 de 15

4. Explicacion de la estructura del proyecto

Capa	Carpeta	Responsabilidad
Rutas	<code>app/routes/</code>	Definen endpoints y codigos HTTP; no contienen logica de negocio
Schemas	<code>app/schemas/</code>	Modelos Pydantic de entrada (<code>UserCreate</code> , <code>UserUpdate</code> , <code>UserPatch</code>) y salida (<code>UserResponse</code>)
Servicios	<code>app/services/</code>	Logica de negocio pura, sin FastAPI
Dependencias	<code>app/dependencies/</code>	Validaciones y busquedas reutilizables inyectadas con <code>Depends()</code>
Datos	<code>app/data/</code>	Base de datos simulada en memoria

Separar estas capas permitio agregar `PUT` , `PATCH` y `DELETE` sin tocar la logica ya existente y mantener cada archivo con una sola responsabilidad.

5. Como se aplico Dependency Injection

Dependencia	Que hace	Donde se usa
<code>get_user_or_404</code>	Busca el usuario y lanza <code>404</code> si no existe	<code>GET</code> , <code>PUT</code> , <code>PATCH</code> , <code>DELETE</code> por ID
<code>validar_email_unico</code>	Impide crear correos duplicados	<code>POST /users</code>
<code>validar_email_unico_en_actualizacion</code>	Impide que un <code>PUT</code> repita el correo de otro usuario	<code>PUT /users/{id}</code>
<code>validar_patch_no_vacio</code>	Devuelve <code>400</code> si el <code>PATCH</code> llega vacio	<code>PATCH /users/{id}</code>
<code>validar_rol_permitido</code>	Valida el rol del query parameter	<code>GET /users</code>
<code>get_api_settings</code>	Entrega la configuracion general	<code>GET /</code>
<code>verificar_api_key</code>	Autenticacion basica simulada por cabecera	disponible para proteger rutas

```
def get_user_or_404(user_id: int = Path(..., ge=1)) -> Dict:
    usuario = user_service.obtener_por_id(user_id)
    if usuario is None:
        raise HTTPException(status_code=404, detail="Usuario no encontrado")
    return usuario

@router.get("/{user_id}", response_model=UserResponse)
def obtener_usuario(usuario: Dict = Depends(get_user_or_404)):
    return UserResponse(**usuario)
```

La busqueda y el `404` se escriben una sola vez y cuatro endpoints los reutilizan.

6. Resumen de pruebas

#	Prueba	Esperado	Resultado
1	GET /users	200	OK
2	GET /users/2	200	OK
3	GET /users/99	404	OK
4	POST /users	201	OK
5	POST correo repetido	400	OK
6	POST datos invalidos	422	OK
7	PUT /users/2	200	OK
8	PUT /users/99	404	OK
9	PATCH /users/3	200	OK
10	PATCH vacio	400	OK
11	DELETE /users/4	200	OK
12	DELETE repetido	404	OK

7. Reflexion final sobre la evolucion del proyecto

La diferencia mas grande frente a la version 1.0 no esta en la cantidad de endpoints sino en la organizacion. Al separar `routes`, `services`, `dependencies` y `data`, cada archivo hace una sola cosa y agregar los nuevos metodos fue casi mecanico.

`Depends()` fue el concepto que mas me sirvio: antes habria repetido el bloque "buscar usuario / si no existe lanzar 404" en cuatro endpoints; ahora vive en una sola funcion. Lo mismo con la validacion de correos y el control del `PATCH` vacio.

Tambien entendi que los codigos de estado son parte del contrato de la API: un `201` al crear, un `400` cuando el cliente envia mal los datos y un `404` cuando el recurso no existe comunican el resultado sin necesidad de leer el mensaje. Unificar el formato de error con un `exception_handler` completa esa idea, porque el cliente siempre recibe la misma estructura.